# Εισαγωγή στη Σχεδίαση Συστημάτων VLSI

# 6η Εργαστηριακή Άσκηση

Ομάδα 11: Βασίλης Κόκκορης (03120867)

Δημήτριος Μιχαλακόπουλος (03120229)

Τμήμα 3 - Βάρδια: 10:45-11:30

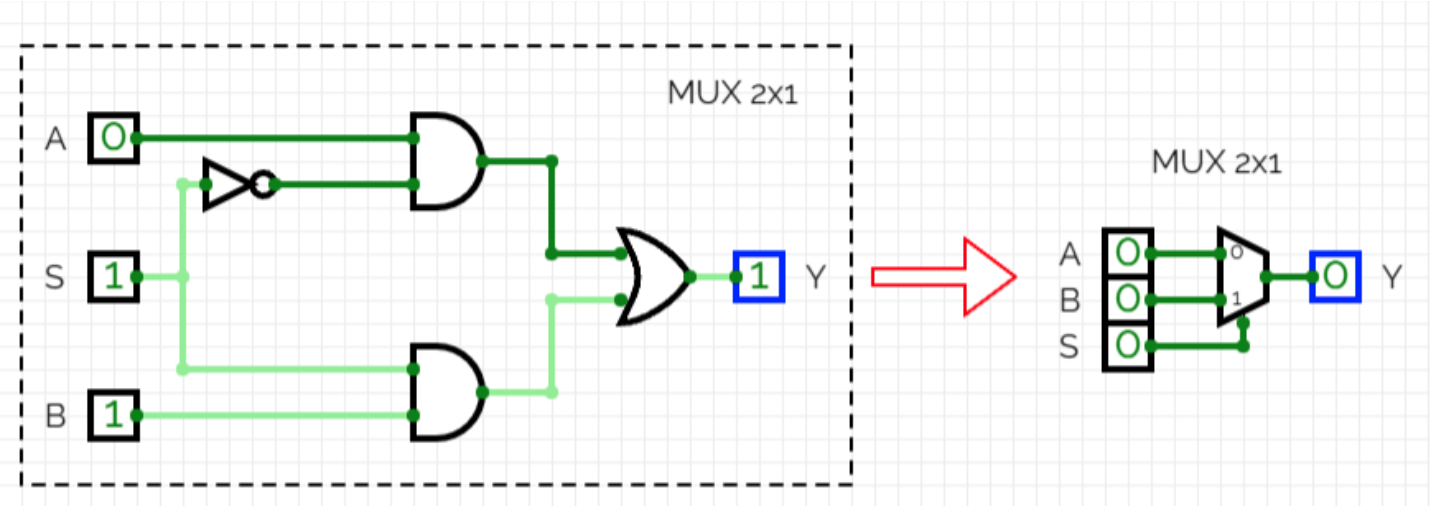
Σχολή: Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

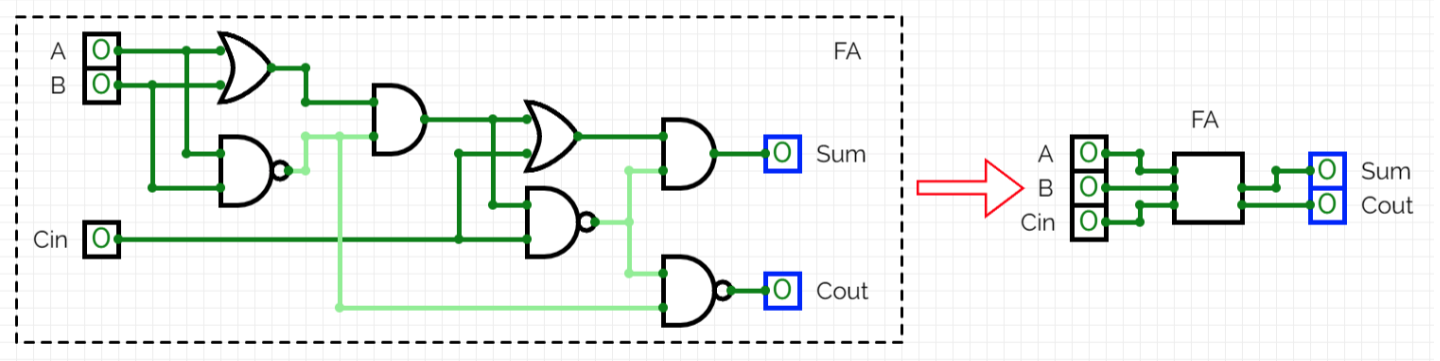
## ΜΕΡΟΣ Α

Η **ιεραρχική περιγραφή** του ζητούμενου κυκλώματος έχει ως εξής:

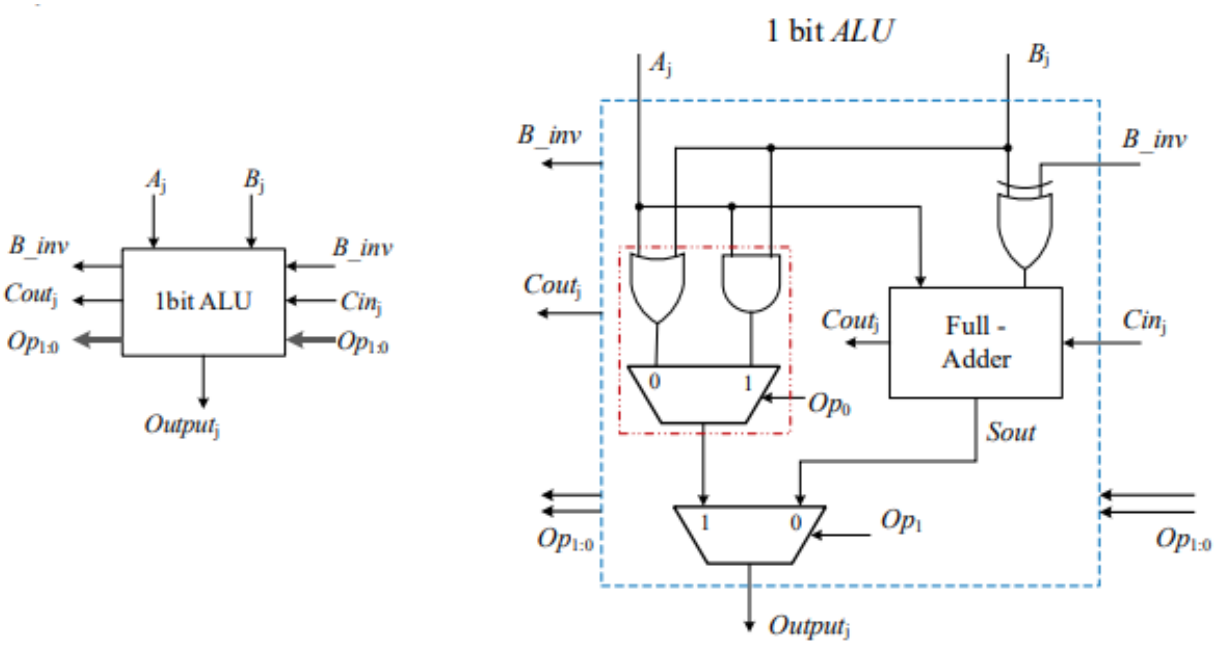
1ο Στάδιο: Σε αυτό το στάδιο βρίσκονται οι λογικές πύλες NOT, AND, OR, NAND, NOR, XOR οι οποίες αποτελούν την βάση του κυκλώματος.

2ο Στάδιο: Σε αυτό το στάδιο βρίσκονται κυκλώματα που προκύπτουν από τις λογικές πύλες του πρώτου σταδίου. Πιο συγκεκριμένα, είναι ο πολυπλέκτης (MUX 2x1) και ο πλήρης αθροιστής (FA). Τα αντίστοιχα κυκλώματα ως «μαύρα κουτιά» φαίνονται παρακάτω:



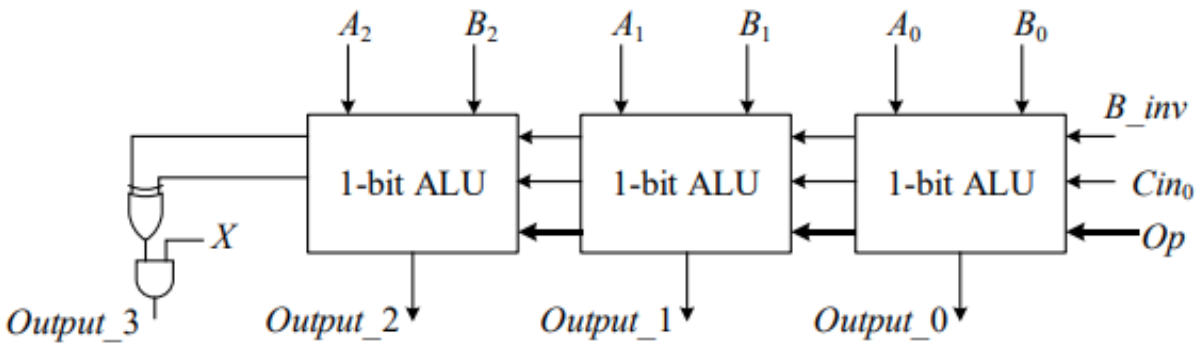



3ο Στάδιο: Σε αυτό το στάδιο χρησιμοποιούμε τις λογικές πύλες του πρώτου σταδίου και τα «μαύρα κουτιά» του δεύτερου σταδίου για να σχεδιάσουμε την αριθμητική και λογική μονάδα ενός bit (1-bit ALU). Το κύκλωμα ως «μαύρο κουτί» φαίνεται παρακάτω:




*Σχήμα 6.1*: 1-bit ALU

4ο Στάδιο: Σε αυτό το στάδιο χρησιμοποιούμε το 1-bit ALU του τρίτου σταδίου και λογικές πύλες από το πρώτο στάδιο για να σχεδιάσουμε την αριθμητική και λογική μονάδα τριών bit (3-bit ALU). Το σχέδιο του τελικού κυκλώματος φαίνεται παρακάτω:




*Σχήμα 6.2:* 3-bit ALU

Ο αντίστοιχος κώδικας Verilog HDL κάθε σταδίου φαίνεται παρακάτω:

1ο Στάδιο:

Πύλες NOT, AND, OR, NAND, NOR, XOR

2ο Στάδιο:

```
module MUX (Y, A, B, S)
        not (Sinv, S);
        pmos (Y, A, S);
        nmos (Y, A, Sinv);
        pmos (Y, B, Sinv);
        nmos (Y, B, S);
endmodule

module FADD (Sum, Cout, A, B, Cin)
        nand (nand1, A, B);
        or (or1, A, B);
        and (and1, or1, nand1);
        nand (nand2, and1, Cin);
        or (or2, and1, Cin);
        and (Sum, nand2, or2);
        nand (Cout, nand2, nand1);
endmodule
```

3ο Στάδιο:

```
module 1bitALU (out, Cout, A, B, Binv, Cin, Op0, Op1)
        nor (nor1, B, Binv);
        and (and1, B, Binv);
        nor (nor2, and1, nor1);
        and (and2, A, B);
        or (or1, A, B);
        MUX (out0, or1, and2, Op0);
        FADD (Sum, Cout, A, nor2, Cin);
        MUX (out, Sum, out0, Op1);
endmodule
```

4ο Στάδιο:

```
module 3bitALU (out0, out1, out2, out3, A0, B0, A1, B1, A2, B2, Op0,
Op1, Cin, Binv, X)
        1bitALU (out0, Cout0, A0, B0, Binv, Cin, Op0, Op1);
        1bitALU (out1, Cout1, A1, B1, Binv, Cout0, Op0, Op1);
        1bitALU (out2, Cout2, A2, B2, Binv, Cout1, Op0, Op1);
        nor (nor1, Op0, Cout2);
        and (and1, Op0, Cout2);
        nor (nor2, and1, nor1);
        and (out3, nor2, X);
endmodule
```

Το Microwind όμως δεν υποστηρίζει αυτή την ιεραρχική υλοποίηση του κυκλώματος σε Verilog HDL, αλλά πρέπει όλος ο κώδικας να βρίσκεται σε ένα module αλλιώς δεν θα μπορέσει να τον κάνει compile.

Ακολουθεί επεξήγηση για την επιλογή κάποιων σημάτων ελέγχου:

- **B_inv=Op0:** Αυτή η επιλογή έγινε γιατί στην πράξη της πρόσθεσης θέλουμε Binv=0 ώστε η XOR να δίνει έξοδο B. Αντίστοιχα στην πράξη της αφαίρεσης θέλουμε Binv=1 ώστε η XOR να δίνει έξοδο B’ για να γίνει η πράξη A+B’+1. Καθώς η κωδικοποίηση του σήματος Op0 είναι Op0=0 για την πρόσθεση και Op0=1 για την αφαίρεση η εξίσωση των δύο σημάτων είναι απολύτως λογική και απλουστεύει το κύκλωμα.

- **Cin0=Op0:** Αυτή η επιλογή έγινε γιατί στην πράξη της πρόσθεσης θέλουμε Cin0=0 και στην πράξη της αφαίρεσης θέλουμε Cin0=1 για να γίνει η πράξη A+B’+1. Καθώς η κωδικοποίηση του σήματος Op0 είναι Op0=0 για την πρόσθεση και Op0=1 για την αφαίρεση η εξίσωση των δύο σημάτων είναι απολύτως λογική και απλουστεύει το κύκλωμα.

- **X=Op1’:** Αυτή η επιλογή έγινε γιατί στην περίπτωση που θέλουμε να εμφανίσουμε το αποτέλεσμα κάποιας λογικής πράξης πρέπει το X=0 ώστε να μηδενίζει το σήμα out3. Αντίστοιχα στην περίπτωση που θέλουμε να εμφανίσουμε το αποτέλεσμα κάποιας αριθμητικής πράξης πρέπει το X=1 ώστε να περνάει το σήμα out3. Καθώς η κωδικοποίηση του σήματος Op1 είναι Op1=0 για τις αριθμητικές πράξεις της πρόσθεσης και της αφαίρεσης, και Op1=1 για τις λογικές πράξεις OR και AND, η εξίσωση των δύο σημάτων είναι απολύτως λογική και απλουστεύει το κύκλωμα.

Ο κώδικας Verilog HDL του κυκλώματος φαίνεται παρακάτω:

```
module ALU3bit(out0,out1,out2,out3,A0,B0,A1,B1,A2,B2,op0,op1);
    input A0,B0,A1,B1,A2,B2,op0,op1;
    output out0,out1,out2,out3;
    wire op0inv,op1inv,z1,z2,z3;
    wire or0,and0,logic0,sum0,cout0,w1,w2,w3,w4,w5,w6,w7,w8;
    wire or1,and1,logic1,sum1,cout1,x1,x2,x3,x4,x5,x6,x7,x8;
    wire or2,and2,logic2,sum2,cout2,y1,y2,y3,y4,y5,y6,y7,y8;

    not (op0inv,op0);
    not (op1inv,op1);

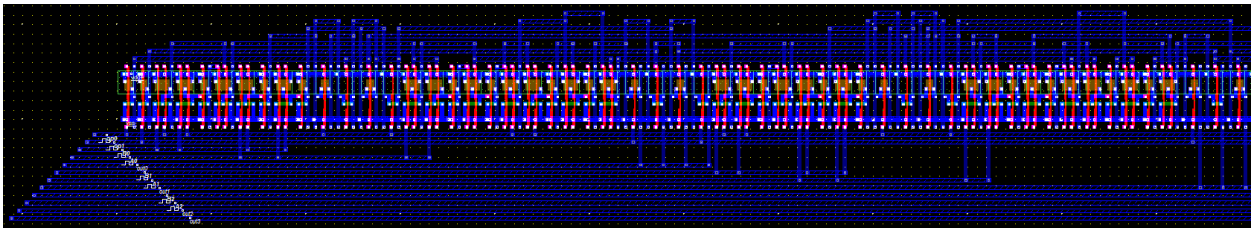
//ALU0
    nor (w1,B0,op0);//Binv XOR
    and (w2,B0,op0);//Binv=op0
    nor (w3,w2,w1);
    or (or0,A0,B0);//Logic OR
    and (and0,A0,B0);//Logic AND
    pmos (logic0,or0,op0);//OR-AND MUX
    nmos (logic0,or0,op0inv);
    pmos (logic0,and0,op0inv);
    nmos (logic0,and0,op0);
    nand (w4,A0,w3);//FULL ADDER
    or (w5,A0,w3);
    and (w6,w5,w4);
    nand (w7,w6,op0);//Cin0=op0
    or (w8,w6,op0);
    and (sum0,w7,w8);
    nand (cout0,w7,w4);
    pmos (out0,sum0,op1);//LOGIC-FA MUX
    nmos (out0,sum0,op1inv);
    pmos (out0,logic0,op1inv);
    nmos (out0,logic0,op1);

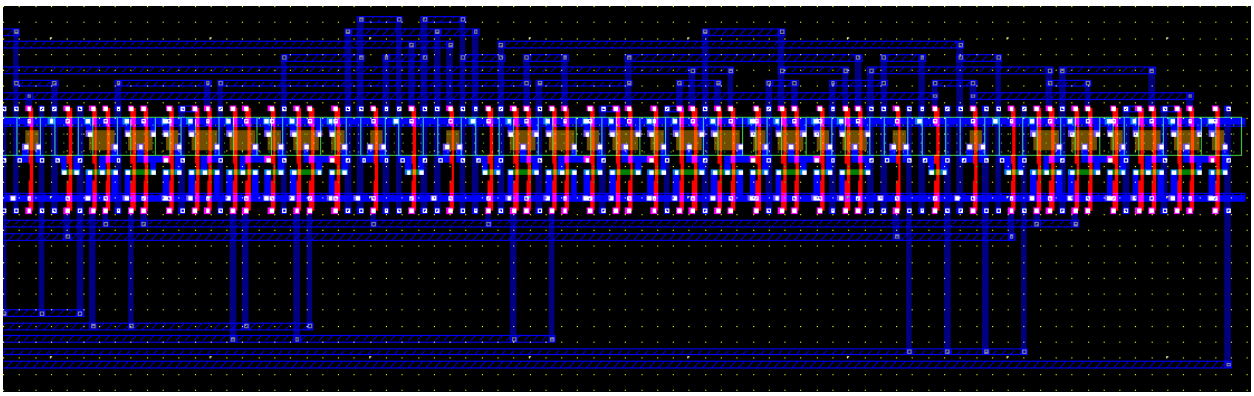
```

```
34      //ALU1
35          nor (x1,B1,op0);//Binv XOR
36          and (x2,B1,op0);//Binv=op0
37          nor (x3,x2,x1);
38          or (or1,A1,B1);//Logic OR
39          and (and1,A1,B1);//Logic AND
40          pmos (logic1,or1,op0);//OR-AND MUX
41          nmos (logic1,or1,op0inv);
42          pmos (logic1,and1,op0inv);
43          nmos (logic1,and1,op0);
44          nand (x4,A1,x3);//FULL ADDER
45          or (x5,A1,x3);
46          and (x6,x5,x4);
47          nand (x7,x6,cout0);//Cin0=cout0
48          or (x8,x6,cout0);
49          and (sum1,x7,x8);
50          nand (cout1,x7,x4);
51          pmos (out1,sum1,op1);//LOGIC-FA MUX
52          nmos (out1,sum1,op1inv);
53          pmos (out1,logic1,op1inv);
54          nmos (out1,logic1,op1);
55
56      //ALU2
57          nor (y1,B2,op0);//Binv XOR
58          and (y2,B2,op0);//Binv=op0
59          nor (y3,y2,y1);
60          or (or2,A2,B2);//Logic OR
61          and (and2,A2,B2);//Logic AND
62          pmos (logic2,or2,op0);//OR-AND MUX
63          nmos (logic2,or2,op0inv);
64          pmos (logic2,and2,op0inv);
65          nmos (logic2,and2,op0);
66          nand (y4,A2,y3);//FULL ADDER
67          or (y5,A2,y3);
68          and (y6,y5,y4);
69          nand (y7,y6,cout1);//Cin0=cout1
70          or (y8,y6,cout1);
71          and (sum2,y7,y8);
72          nand (cout2,y7,y4);
73          pmos (out2,sum2,op1);//LOGIC-FA MUX
74          nmos (out2,sum2,op1inv);
75          pmos (out2,logic2,op1inv);
76          nmos (out2,logic2,op1);
77
```

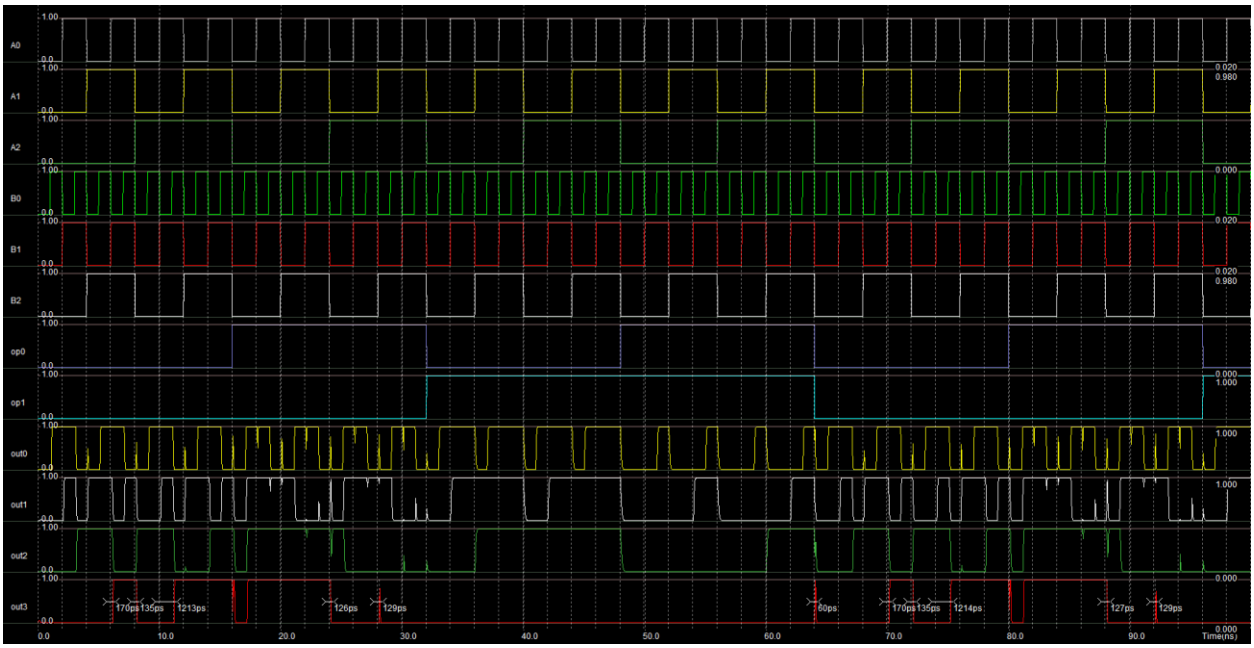
```
78      //out3
79          nor (z1,op0,cout2);
80          and (z2,op0,cout2);
81          nor (z3,z2,z1);
82          and (out3,z3,op1inv);//X=op1inv
83      endmodule
84
85      // Simulation parameters in Verilog Format
86      always
87      #400 A0=~A0;
88      #200 B0=~B0;
89      #800 A1=~A1;
90      #400 B1=~B1;
91      #1600 A2=~A2;
92      #800 B2=~B2;
93      #3200 op0=~op0;
94      #6400 op1=~op1;
95
96      // Simulation parameters
97      // A0 CLK 2 2
98      // B0 CLK 1 1
99      // A1 CLK 4 4
100     // B1 CLK 2 2
101     // A2 CLK 8 8
102     // B2 CLK 4 4
103     // op0 CLK 16 16
104     // op1 CLK 32 32
```

Το layout του κυκλώματος στο Microwind φαίνεται παρακάτω:





Η προσομοίωση του κυκλώματος φαίνεται παρακάτω:




<u>Σχόλια:</u> Παρακάτω βρίσκονται κάποιες ενδεικτικές τιμές προκειμένου να επαληθεύσουμε αν είναι σωστή η υλοποίησή μας:

Για πρόσθεση (Op1Op0=00), παίρνουμε A2A1A0=111 και B2B1B0=110, δηλαδή έχουμε την πράξη 7+6=13. Η έξοδος που παίρνουμε είναι out3out2out1out0=1101, δηλαδή το 13, άρα η πρόσθεση λειτουργεί.

Για αφαίρεση (Op1Op0=01), παίρνουμε A2A1A0=100 και B2B1B0=001, δηλαδή έχουμε την πράξη 4-1=3. Η έξοδος που παίρνουμε είναι out3out2out1out0=0011, δηλαδή το 3, άρα η αφαίρεση λειτουργεί.

Για OR (Op1Op0=10), παίρνουμε A2A1A0=110 και B2B1B0=100, και κάνουμε για κάθε bit την λογική πράξη OR. Η έξοδος που παίρνουμε είναι out3out2out1out0=0110, άρα η OR λειτουργεί.

Για AND (Op1Op0=11), παίρνουμε A2A1A0=011 και B2B1B0=110, και κάνουμε για κάθε bit την λογική πράξη AND. Η έξοδος που παίρνουμε είναι out3out2out1out0=0010, άρα η AND λειτουργεί.

Τελικά, λοιπόν, το κύκλωμα μας λειτουργεί όπως επιθυμούμε.

## ΜΕΡΟΣ Β

Κάνουμε compile το αρχείο “And.v” στο Microwind σε τεχνολογία “cmos65n.rul” με ονομαστικές τιμές $V_{dd}$=1V, A[0]=1ns, A[1]=2ns και τοποθετούμε έναν πυκνωτή χωρητικότητας C=0.01pF στην έξοδο.

1. Σε αυτό το ερώτημα θέλουμε να προσομοιώσουμε το κύκλωμα για διάφορες συχνότητες και να βρούμε την ισχύ σε κάθε περίπτωση. Έτσι:

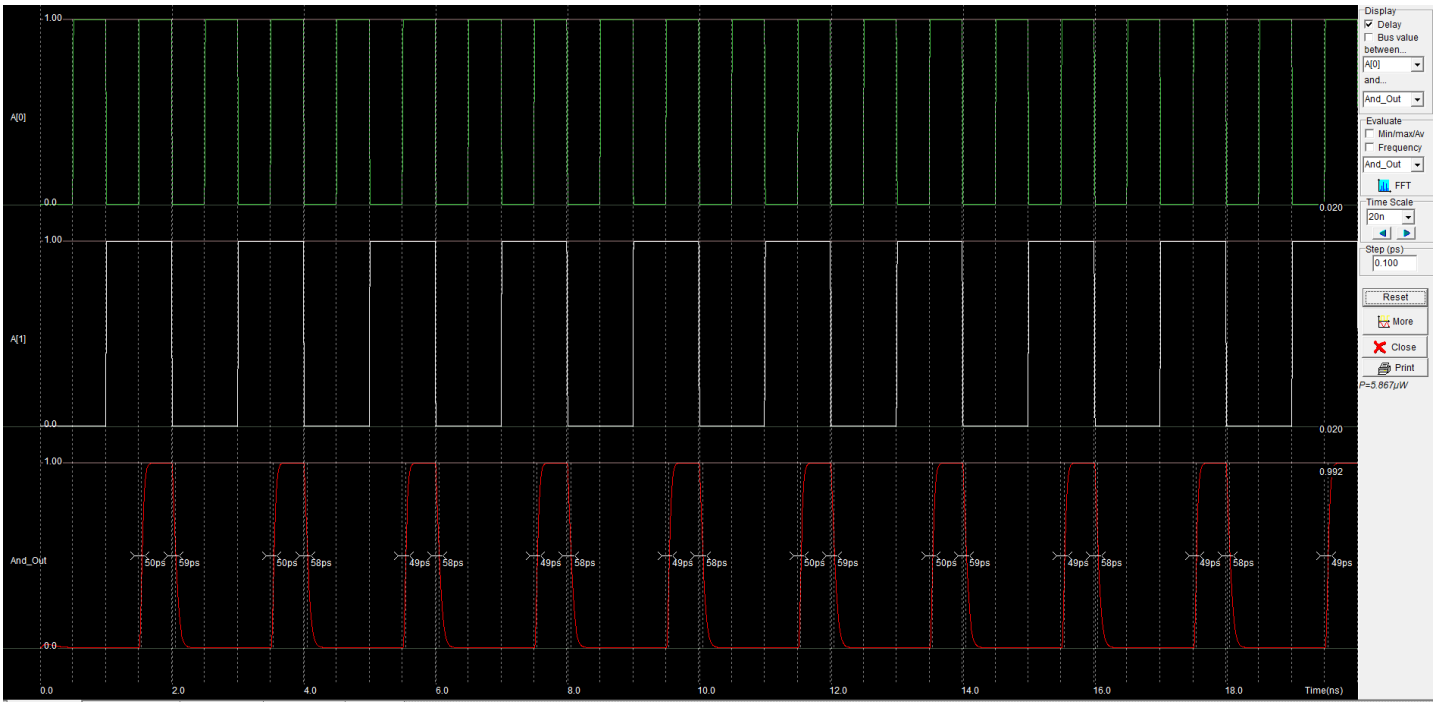
<u>1η Περίπτωση</u>

A[0]=tl+tr+th+tf=0.49+0.01+0.49+0.01=1ns

A[1]=tl+tr+th+tf=0.99+0.01+0.99+0.01=2ns

Παίρνουμε ισχύ P=**5.867μW**

Η προσομοίωση του κυκλώματος φαίνεται παρακάτω:



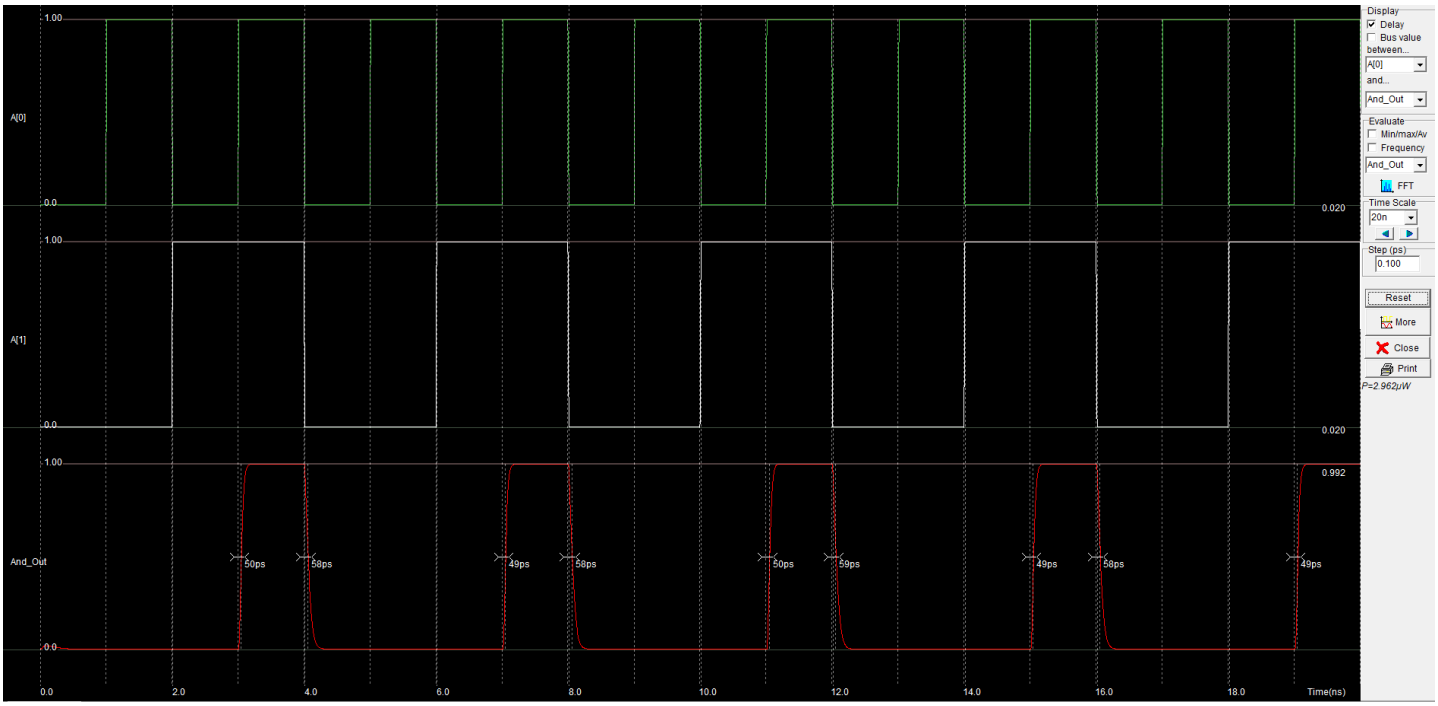
2η Περίπτωση:

A[0]=tl+tr+th+tf=0.99+0.01+0.99+0.01=2ns

A[1]=tl+tr+th+tf=1.99+0.01+1.99+0.01=4ns

Παίρνουμε ισχύ **P=2.962μW**

Η προσομοίωση του κυκλώματος φαίνεται παρακάτω:



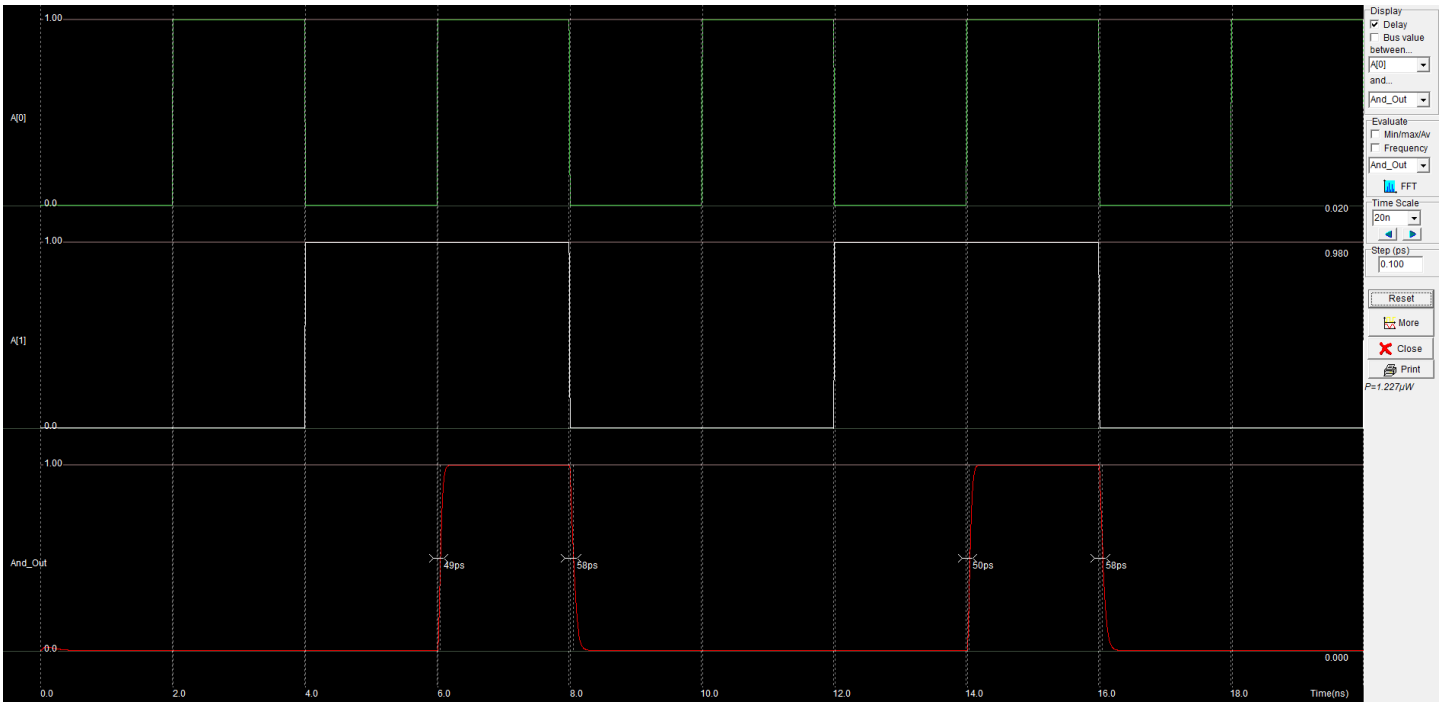
3η Περίπτωση:

A[0]=tl+tr+th+tf=1.99+0.01+1.99+0.01=4ns

A[1]=tl+tr+th+tf=3.99+0.01+3.99+0.01=8ns

Παίρνουμε ισχύ **P=1.227μW**

Η προσομοίωση του κυκλώματος φαίνεται παρακάτω:



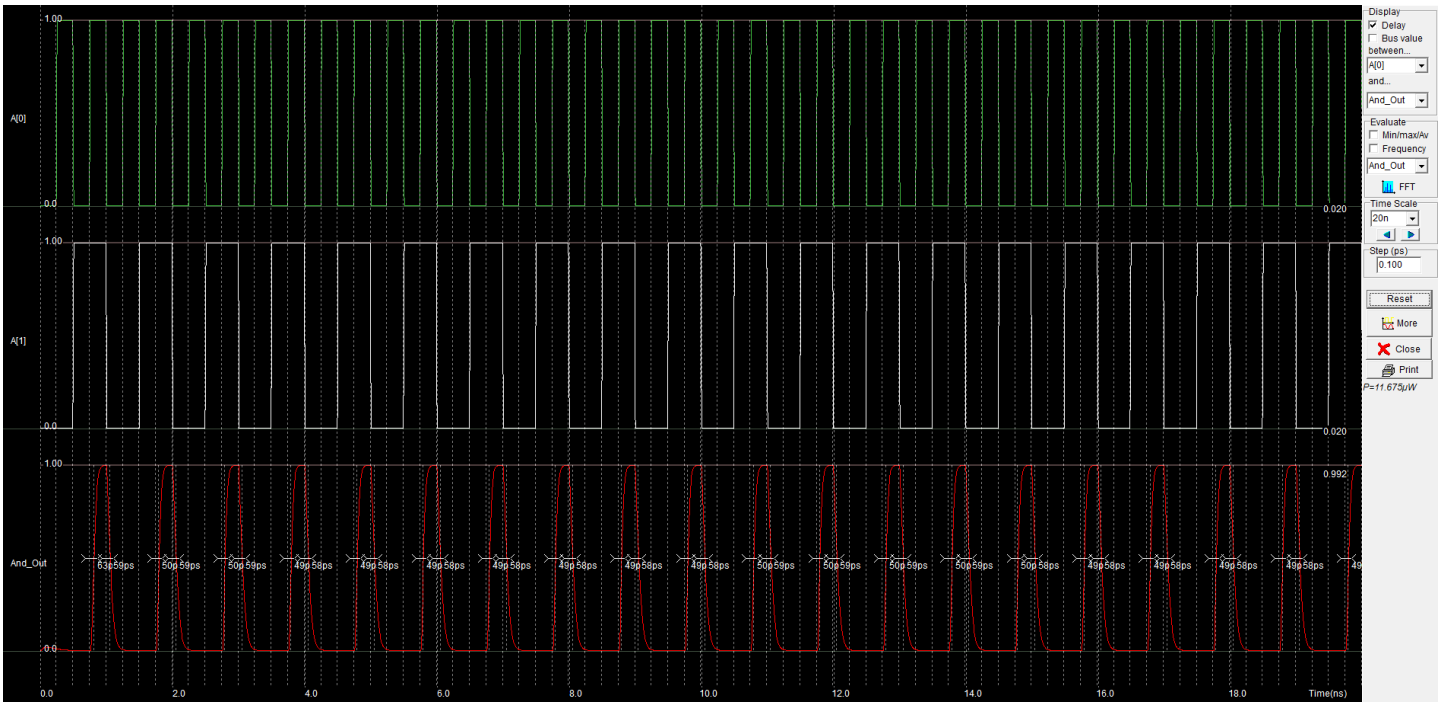
4η Περίπτωση:

A[0]=tl+tr+th+tf=0.24+0.01+0.24+0.01=0.5ns

A[1]=tl+tr+th+tf=0.49+0.01+0.49+0.01=1ns

Παίρνουμε ισχύ **P=11.675μW**

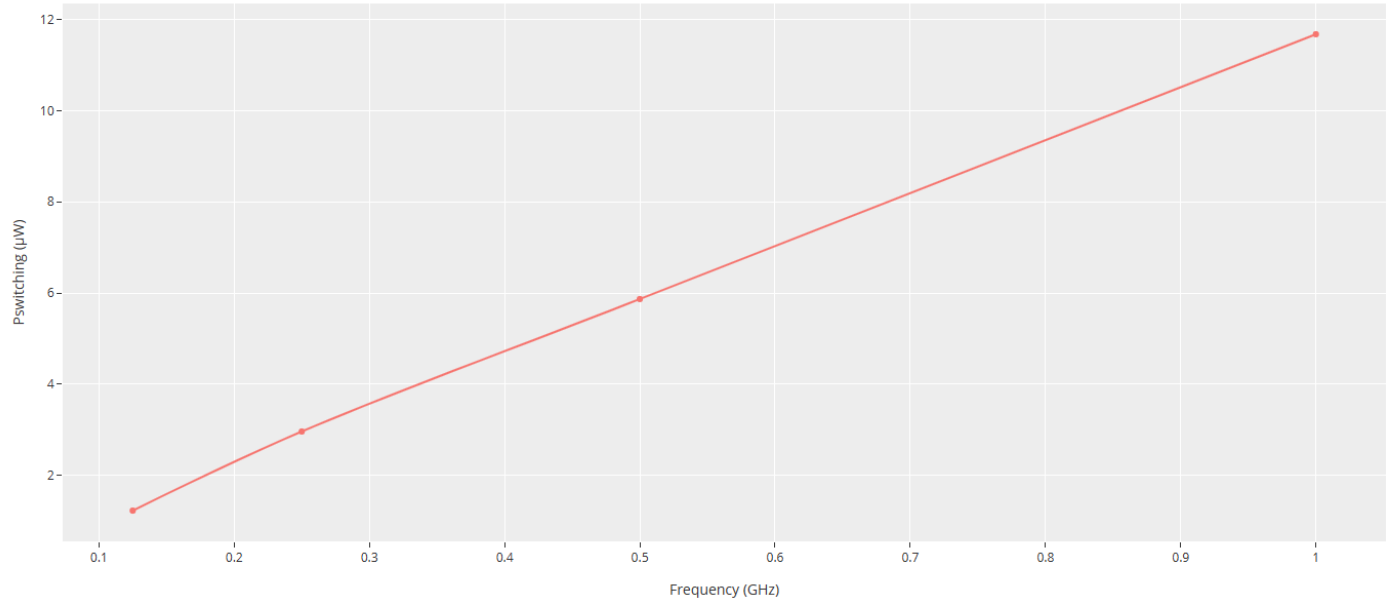
Η προσομοίωση του κυκλώματος φαίνεται παρακάτω:



Ο συγκεντρωτικός πίνακας τιμών που προκύπτει είναι ο εξής:

| Συχνότητα A[0] (GHz) | Ισχύς (μW) |
|---|---|
| 0.125 | 1.227 |
| 0.25 | 2.962 |
| 0.5 | 5.867 |
| 1 | 11.675 |

Η γραφική παράσταση της ισχύος συναρτήσει της συχνότητας του A[0] φαίνεται παρακάτω:




Παρατηρούμε ότι όσο αυξάνεται η συχνότητα στο ρολόι της εισόδου A[0], τόσο αυξάνεται και η ισχύς που καταναλώνεται από το κύκλωμα. Η σχέση, μάλιστα, μεταξύ αυτών των δύο μεγεθών είναι γραμμική, συμπέρασμα που έιναι λογικό καθώς η σχέση μεταξύ ισχύος και συχνότητας είναι:

$$P_{total} = P_{dynamic} + P_{static} = P_{switching} + P_{short-circuit} + P_{static}$$

$$\xrightarrow{P_{static}=0} P_{total} = a * C * {V_{DD}}^2 * \boldsymbol{f}$$

2. Σε αυτό το ερώτημα θέλουμε να προσομοιώσουμε το κύκλωμα με A[0]=1ns απόλυτο τετραγωνικό παλμό και A[1] περίπου 2ns για σταθερές τιμές $t_{low}$ και $t_{high}$, και διαφορετικές τιμές $t_{rise}$ και $t_{fall}$. Έτσι:

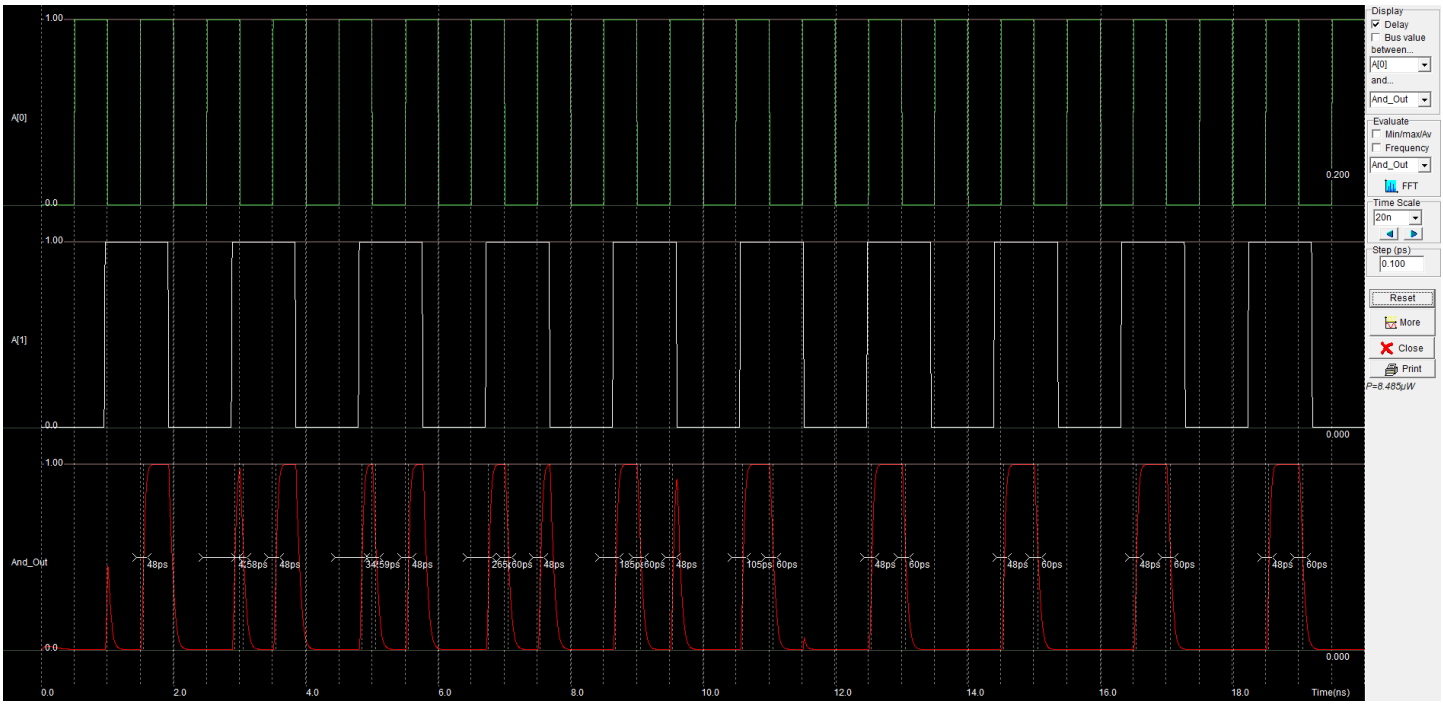
<u>1η Περίπτωση</u>

A[0]=tl+tr+th+tf=0.499+0.001+0.499+0.01=1ns

A[1]=tl+tr+th+tf=0.95+0.01+0.95+0.01=1.92ns

Παίρνουμε ισχύ P=**8.485μW**

Η προσομοίωση του κυκλώματος φαίνεται παρακάτω:



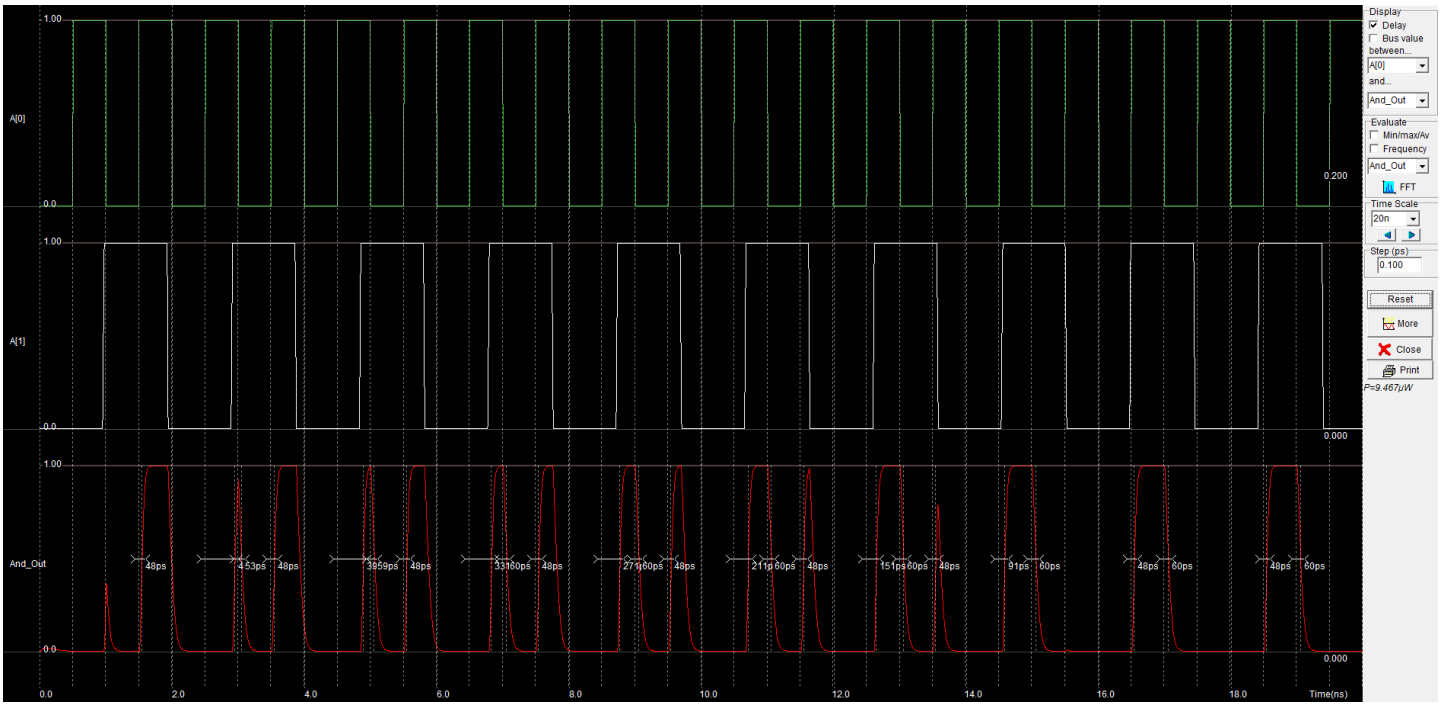
2η Περίπτωση

A[0]=tl+tr+th+tf=0.499+0.001+0.499+0.01=1ns

A[1]=tl+tr+th+tf=0.95+0.02+0.95+0.02=1.94ns

Παίρνουμε ισχύ P=**9.467μW**

Η προσομοίωση του κυκλώματος φαίνεται παρακάτω:



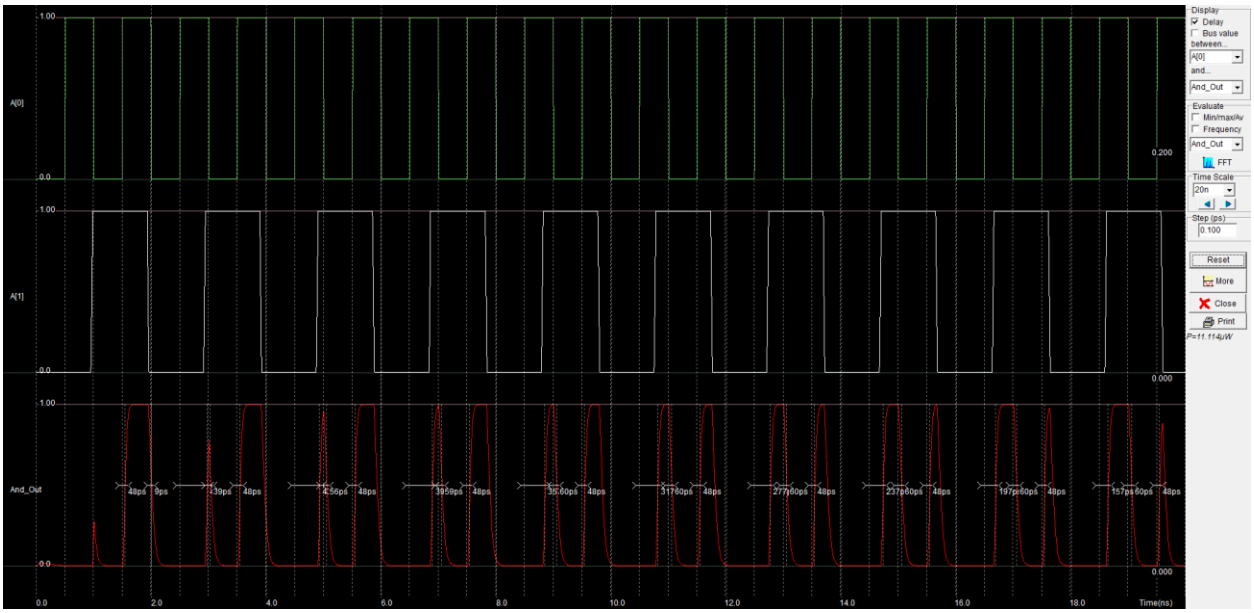
3η Περίπτωση

A[0]=tl+tr+th+tf=0.499+0.001+0.499+0.01=1ns

A[1]=tl+tr+th+tf=0.95+0.03+0.95+0.03=1.96ns

Παίρνουμε ισχύ P=**11.114μW**

Η προσομοίωση του κυκλώματος φαίνεται παρακάτω:



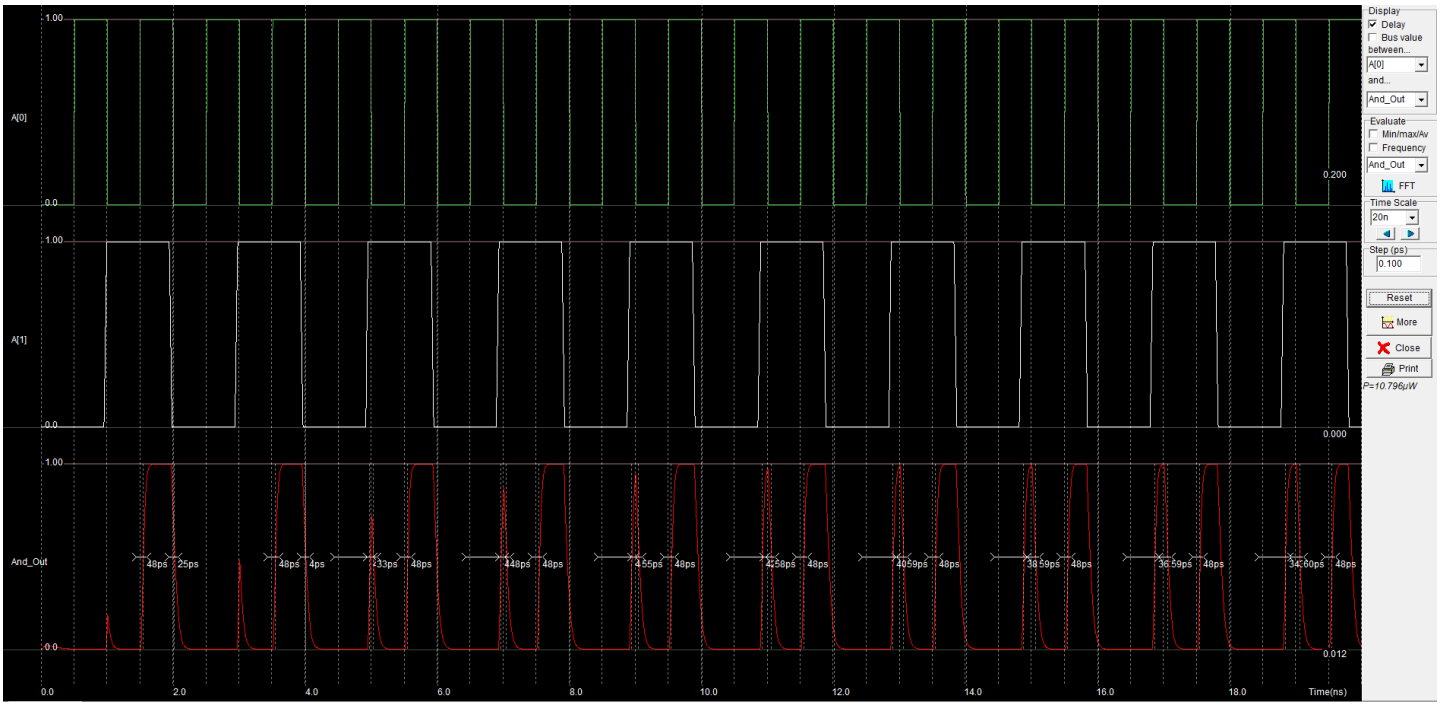
4η Περίπτωση

A[0]=tl+tr+th+tf=0.499+0.001+0.499+0.01=1ns

A[1]=tl+tr+th+tf=0.95+0.04+0.95+0.04=1.98ns

Παίρνουμε ισχύ P=**10.796μW**

Η προσομοίωση του κυκλώματος φαίνεται παρακάτω:



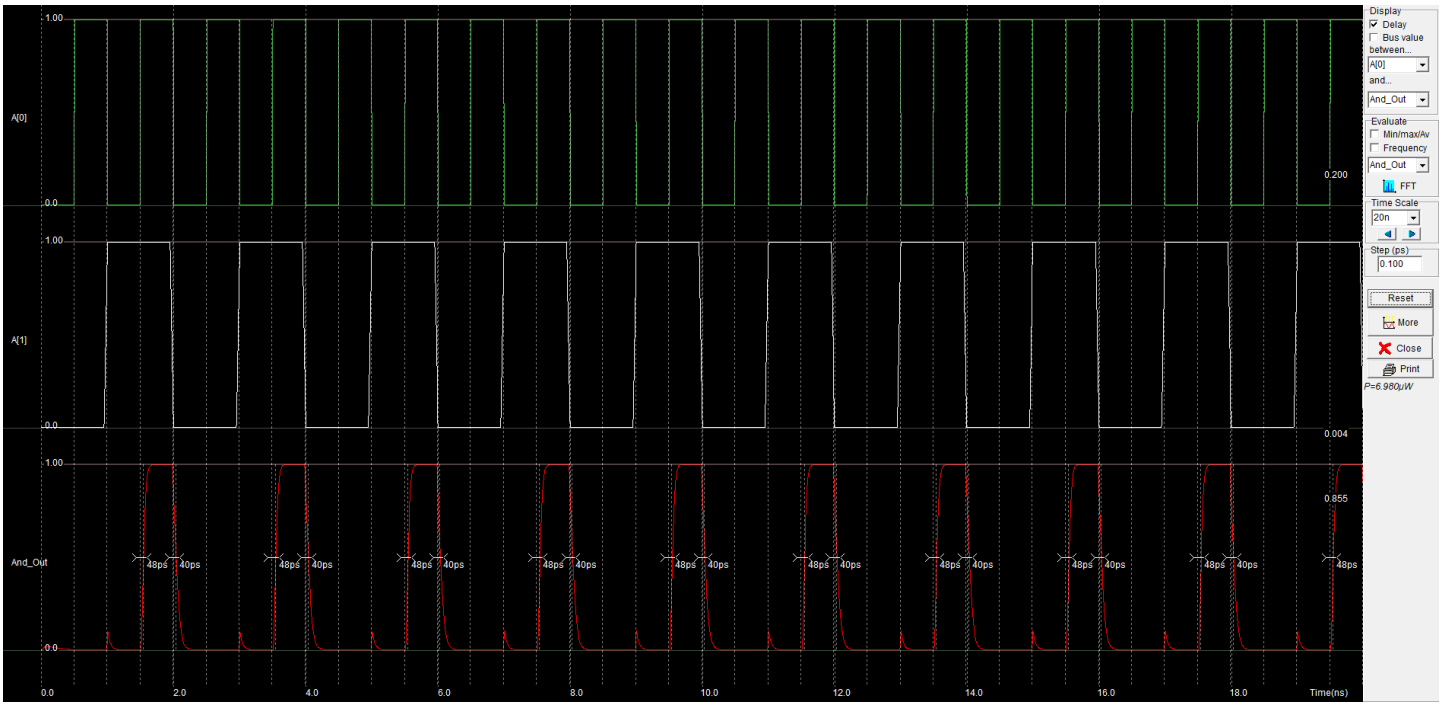
5η Περίπτωση

A[0]=tl+tr+th+tf=0.499+0.001+0.499+0.01=1ns

A[1]=tl+tr+th+tf=0.95+0.05+0.95+0.05=2ns

Παίρνουμε ισχύ P=**6.980μW**

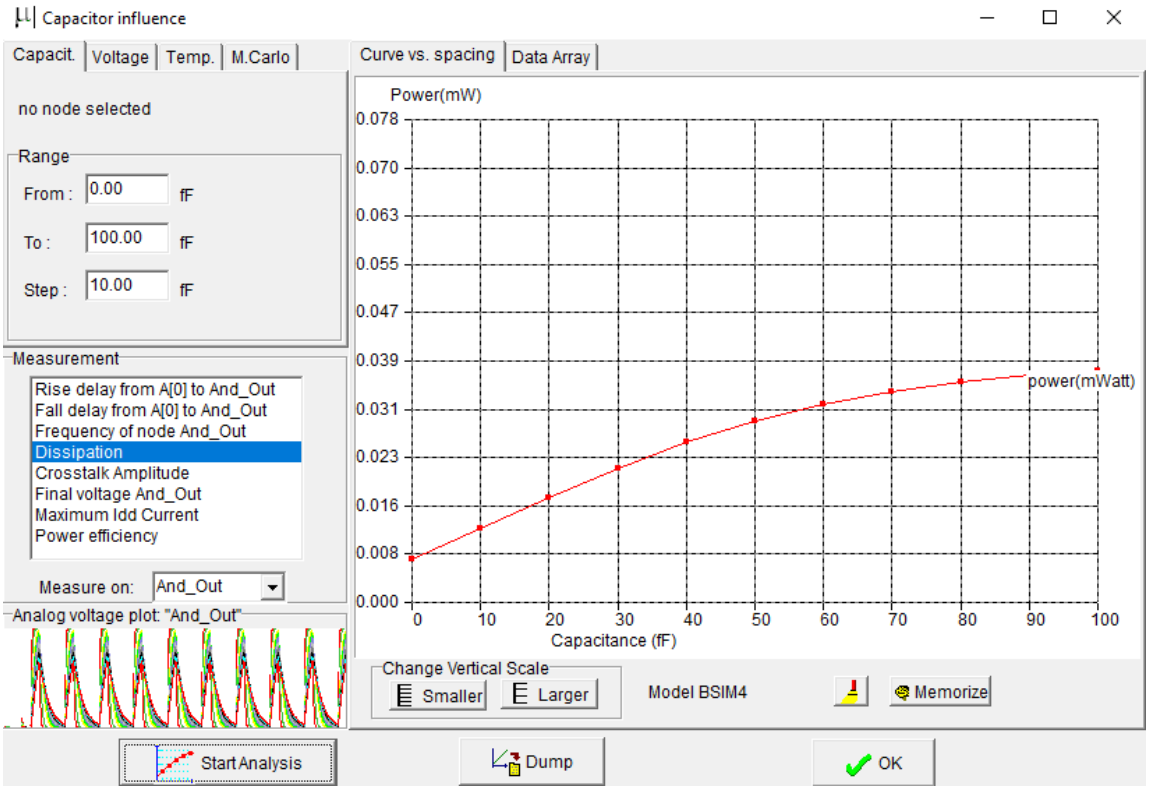
Η προσομοίωση του κυκλώματος φαίνεται παρακάτω:




Παρατηρούμε ότι σε αυτή την περίπτωση δεν υπάρχει κάποια σχέση μεταξύ της μεταβολής της ισχύος και την αύξηση της σηχνότητας της εξόδου A[1]. Εδώ η διαφορά παρατηρείται στον συχγρονισμό που έχουν τα ρολόγια των δύο εισόδων σε κάθε περίπτωση. Όταν αυτά είναι κακώς συχγρονισμένα βλέπουμε έντοντα spikes μικρής διάρκειας αλλά μεγάλης τάσης στην έξοδο της πύλης AND, τα οποία προφανώς ευθύνονται για την αντίστοιχη αύξηση της ίσχύος. Από τις περιπτώσεις που προσομοιώσαμε, επομένως, καλύτερο συγχρονισμό εισόδων έχει το κύκλωμα με $t_r$=$t_f$=0.05ns καθώς έχει την μικρότερη ισχύ P=6.98μW.

3. Σε αυτό το ερώτημα θέλουμε να μετρήσουμε την κατανάλωση ισχύος του κυκλώματος για διάφορες τιμές του πυκνωτή εκτελώντας μια sweep analysis από την λειτουργία “Dissipation“.

Η γραφική παράσταση της ισχύος συνερτήσει του φορτίου είναι η εξής:




| Capacitance (fF) | Power (mW) |
|---|---|
| 0.000 | 0.007 |
| 10.000 | 0.012 |
| 20.000 | 0.017 |
| 30.000 | 0.022 |
| 40.000 | 0.026 |
| 50.000 | 0.029 |
| 60.000 | 0.032 |
| 70.000 | 0.034 |
| 80.000 | 0.036 |
| 90.000 | 0.037 |
| 100.000 | 0.038 |

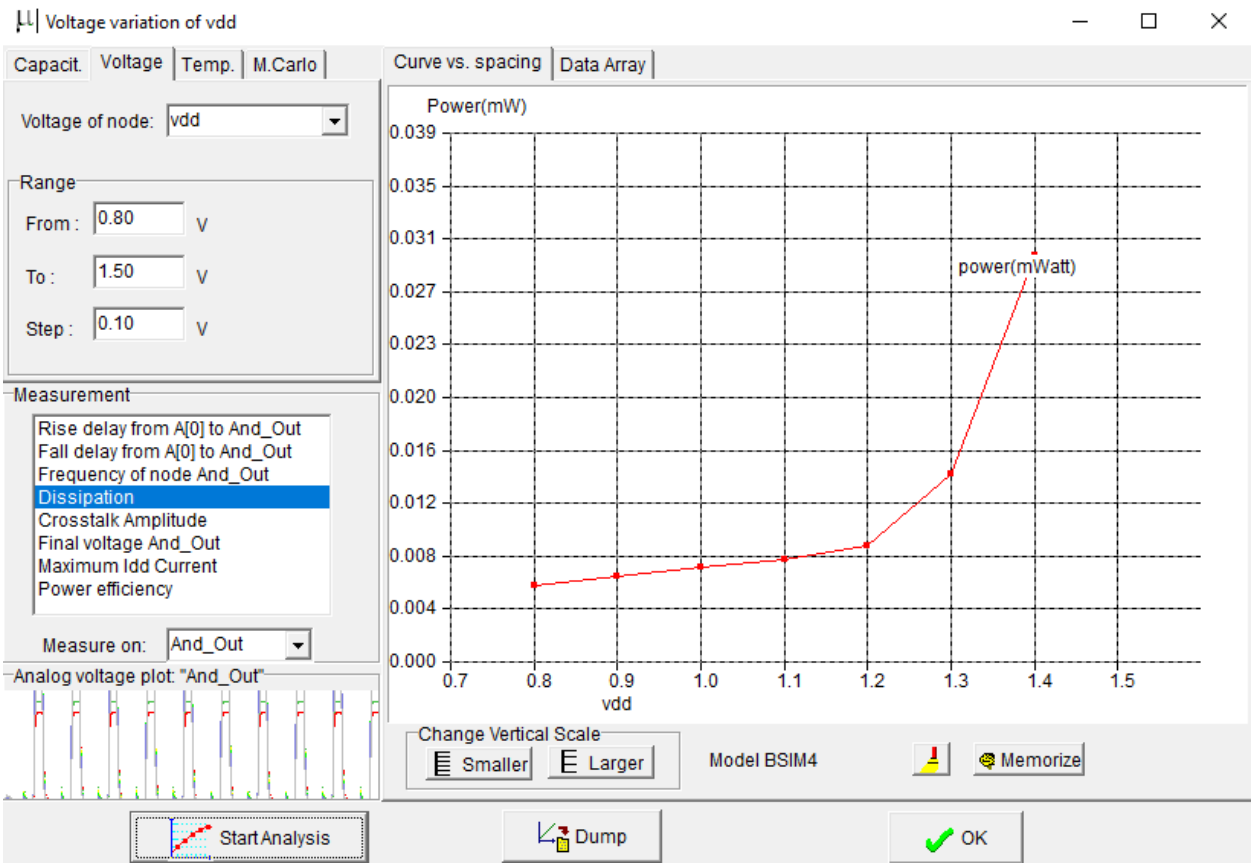
Παρατηρούμε ότι η τιμή της ισχύος αυξάνεται γραμμικά σε σχέση με την τιμή του πυκνωτή, τείνωντας όμως να σταθεροποιηθεί σε μία μέγιστη τιμή. Αυτή η γραμμικότητα επιβεβαιώνεται από τον τύπο που υπολογίζει την συνολική ισχύ, όπως φαίνεται παρακάτω:

$$P_{total} = P_{dynamic} + P_{static} = P_{switching} + P_{short-circuit} + P_{static}$$

$$\xrightarrow{P_{static}=0} P_{total} = a * \boldsymbol{C} * {V_{DD}}^2 * f$$

4. Σε αυτό το ερώτημα θέλουμε να μετρήσουμε την κατανάλωση ισχύος του κυκλώματος για διάφορες τιμές της τάσης τροφοδοσίας εκτελώντας μια sweep analysis από την λειτουργία “Dissipation“.

Η γραφική παράσταση της ισχύος συνερτήσει της τάσης είναι η εξής:



| Vdd (V) | Power (mW) |
|---|---|
| 0.800 | 0.006 |
| 0.900 | 0.006 |
| 1.000 | 0.007 |
| 1.100 | 0.008 |
| 1.200 | 0.009 |
| 1.300 | 0.014 |
| 1.400 | 0.030 |

Παρατηρούμε ότι η τιμή της ισχύος αυξάνεται τετραγωνικά σε σχέση με την τιμή της τάσης τροφοδοσίας. Όπως και πρίν η παρατήρηση αυτή επιβεβαιώνεται από τον τύπο που υπολογίζει την συνολική ισχύ, όπως φαίνεται παρακάτω:

$$P_{total} = P_{dynamic} + P_{static} = P_{switching} + P_{short-circuit} + P_{static}$$

$$\overset{P_{static}=0}{\Longrightarrow} P_{total} = a * C * \boldsymbol{V_{DD}}^{\mathbf{2}} * f$$